

OOPJ CH 2.

1. નીચેના terms ની વ્યાખ્યા આપો:

a) Abstraction (એબ્સ્ટ્રેક્શન)

Abstraction નો અર્થ છે અનાવશ્યક વિગતો છુપાવીને માત્ર જરૂરી માહિતી બતાવવી.

પ્રોગ્રામિંગમાં user ને માત્ર એટલું જ બતાવવામાં આવે છે જેટલું કામ માટે જરૂરી હોય, અંદરની કાર્યપદ્ધતિ છુપાવી દેવામાં આવે છે.

ઉદાહરણ:

કાર ચલાવતી વખતે આપણને engine કેવી રીતે કામ કરે છે તે જાણવાની જરૂર નથી; માત્ર steering, brake, accelerator વાપરવા પડે છે.

Java માં abstraction કેવી રીતે મળે?

- abstract class દ્વારા
- interface દ્વારા

ફાયદા:

- code સરળ બને છે
- security વધે છે
- complexity ઓછી થાય છે

b) Encapsulation (એન્કેપ્સ્યુલેશન)

Encapsulation એટલે **data** અને **methods** ને એક જ **unit (class)** માં બાંધવી અને data ને direct access થી બચાવવું.

બીજા શબ્દોમાં, class ના variables ને private રાખી ને તેમની access માટે getter અને setter methods વાપરવામાં આવે છે.

ઉદાહરણ:

```
class Student {
```

```
private int age;

public void setAge(int a) {
    age = a;
}

public int getAge() {
    return age;
}
}
```

ફાયદા:

- data સુરક્ષિત રહે છે
- direct access રોકાય છે
- code maintenance સરળ બને છે

c) Inheritance (ઈન્હેરિટન્સ)

Inheritance એટલે એક class બીજી class ની **properties** અને **methods** ને inherit કરે.

જે class થી વારસો મળે તેને **parent class / super class** કહે છે અને જે class વારસો લે તેને **child class / sub class** કહે છે.

ઉદાહરણ:

```
class A {
    void show() {
        System.out.println("Parent class");
    }
}

class B extends A {
    void display() {
        System.out.println("Child class");
    }
}
```

ફાયદા:

- code reuse થાય છે
 - time બચાવે છે
 - code સરળ બને છે
-

d) Polymorphism (પોલીમોર્ફિઝમ)

Polymorphism નો અર્થ છે એક જ નામ, પણ અલગ-અલગ સ્વરૂપ.

Java માં polymorphism મુખ્યત્વે બે પ્રકારનો હોય છે:

1. **Compile-time polymorphism** – method overloading
2. **Run-time polymorphism** – method overriding

ઉદાહરણ:

એક જ `print()` method અલગ પ્રકારના input સાથે અલગ પરિણામ આપી શકે.

ફાયદા:

- code flexible બને છે
 - same method name નો ઉપયોગ થઈ શકે છે
 - extensibility વધે છે
-

e) Class (ક્લાસ)

Class એ એક **blueprint / template** છે જેના આધારે object બનાવવામાં આવે છે.

Class માં:

- data members (variables)
- member functions (methods)

હોય છે.

ઉદાહરણ:

```
class Car {  
    String color;  
    void drive() {  
        System.out.println("Car is driving");  
    }  
}
```

અહીં Car એક class છે.

f) Object (ઓબ્જેક્ટ)

Object એ class નો **real world instance** છે.

Class માત્ર design છે, જ્યારે object actual memory માં બને છે અને તેની values રાખે છે.

ઉદાહરણ:

```
Car c1 = new Car();
```

અહીં c1 object છે.

Object ની ખાસિયતો:

- state (values)
- behavior (methods)
- identity

2. Constructor ની વ્યાખ્યા આપો. Types of constructors example સાથે સમજાવો.

Constructor શું છે?

Constructor એ એક ખાસ method છે જે object બનાવતી વખતે automatic call થાય છે. Constructor નું કામ object ને initial values આપવાનું છે.

Constructor ની ખાસિયતો:

- તેનું નામ class જેવું જ હોય છે
- તેમાં કોઈ return type નથી
- object create થતા જ call થાય છે
- initialization માટે વપરાય છે

Constructor ની પ્રકાર

1) Default Constructor

જો programmer constructor ન લખે, તો Java આપમેળે default constructor આપે છે.

ઉદાહરણ:

```
class Demo {
    Demo() {
        System.out.println("Default Constructor");
    }

    public static void main(String[] args) {
        Demo d = new Demo();
    }
}
```

Output:

Default Constructor

2) Parameterized Constructor

જો constructor માં parameters હોય તેને parameterized constructor કહે છે.

ઉદાહરણ:

```
class Student {
    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }

    void display() {
        System.out.println(id + " " + name);
    }

    public static void main(String[] args) {
        Student s1 = new Student(101, "Rahul");
        s1.display();
    }
}
```

Output:

101 Rahul

3) Copy Constructor

Java માં built-in copy constructor નથી, પણ programmer પોતે એવું constructor બનાવી શકે છે જે બીજા object ની values copy કરે.

ઉદાહરણ:

```
class Student {
    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }

    Student(Student s) {
        id = s.id;
        name = s.name;
    }

    void display() {
        System.out.println(id + " " + name);
    }

    public static void main(String[] args) {
        Student s1 = new Student(101, "Aman");
        Student s2 = new Student(s1);

        s1.display();
        s2.display();
    }
}
```

3. Constructor Overloading example સાથે સમજાવો.

Constructor Overloading શું છે?

એક જ class માં અલગ-અલગ parameters વાળા બહુ constructors બનાવવામાં આવે તેને **constructor overloading** કહે છે.

Example:

```
class Box {
    int length, width, height;
```

```

Box() {
    length = 0;
    width = 0;
    height = 0;
}

Box(int l, int w, int h) {
    length = l;
    width = w;
    height = h;
}

Box(int side) {
    length = side;
    width = side;
    height = side;
}

void display() {
    System.out.println(length + " " + width + " " + height);
}

public static void main(String[] args) {
    Box b1 = new Box();
    Box b2 = new Box(10, 20, 30);
    Box b3 = new Box(5);

    b1.display();
    b2.display();
    b3.display();
}
}

```

સમજાવટ:

- Box() → no argument constructor
- Box(int l, int w, int h) → three arguments
- Box(int side) → one argument

ફાયદો:

એક class માં અલગ-અલગ રીતે object initialize કરી શકાય છે.

4. static keyword example સાથે સમજાવો.

static keyword શું છે?

static keyword એ class-level member બનાવવા માટે વપરાય છે.

એનો અર્થ એ કે static variable, method અથવા block class સાથે જોડાયેલ હોય છે, object સાથે નહીં.

static variable

એક shared variable હોય છે જે class ના બધા objects માટે common હોય છે.

static method

Object બનાવ્યા વગર call કરી શકાય છે.

static block

Class load થતા જ execute થાય છે.

Example:

```
class Demo {
    static int count = 0;

    Demo() {
        count++;
        System.out.println(count);
    }

    static void show() {
        System.out.println("Static method");
    }

    public static void main(String[] args) {
        Demo d1 = new Demo();
        Demo d2 = new Demo();
        Demo.show();
    }
}
```

Output:

```
1
2
Static method
```

સમજાવટ:

- count static હોવાથી shared છે
- show() method ને object વગર call કર્યું

5. Method Overloading example સાથે સમજાવો.

Method Overloading શું છે?

એક જ class માં એ જ **method name** સાથે અલગ-અલગ parameters વાળી methods બનાવવી તેને method overloading કહે છે.

Rules:

- method name same હોય
- parameters અલગ હોવા જોઈએ
- return type અલગ હોય તો પણ overload count નહીં થાય જો parameters same હોય

Example:

```
class MathDemo {  
  
    void add(int a, int b) {  
        System.out.println(a + b);  
    }  
  
    void add(int a, int b, int c) {  
        System.out.println(a + b + c);  
    }  
  
    void add(double a, double b) {  
        System.out.println(a + b);  
    }  
  
    public static void main(String[] args) {  
        MathDemo m = new MathDemo();  
        m.add(10, 20);  
        m.add(10, 20, 30);  
        m.add(5.5, 2.5);  
    }  
}
```

Output:

```
30  
60  
8.0
```

ફાયદા:

- same નામથી related કામ થઈ શકે
- code readable બને છે
- flexibility વધે છે

6. નીચેના string functions example સાથે સમજાવો:

`length()`, `charAt()`, `substring()`, `toUpperCase()`, `toLowerCase()`

Java String class માં ઘણા useful methods છે.

1) length()

String માં કેટલા characters છે તે આપે છે.

Example:

```
String s = "Java";  
System.out.println(s.length());
```

Output:

4

2) charAt()

આ method આપેલા index પરનો character આપે છે.

Example:

```
String s = "Java";  
System.out.println(s.charAt(1));
```

Output:

a

Note: index 0 થી શરૂ થાય છે.

3) substring()

String નો ભાગ (part) return કરે છે.

Example 1:

```
String s = "Programming";  
System.out.println(s.substring(3));
```

Output:

gramming

Example 2:

```
String s = "Programming";  
System.out.println(s.substring(0, 7));
```

Output:

Program

4) toUpperCase()

String ના અધી characters ને capital letters માં બદલે છે.

Example:

```
String s = "hello";  
System.out.println(s.toUpperCase());
```

Output:

HELLO

5) toLowerCase()

String ના અધી characters ને small letters માં બદલે છે.

Example:

```
String s = "JAVA";  
System.out.println(s.toLowerCase());
```

Output:

```
java
```

Combined Example:

```
class StringDemo {  
    public static void main(String[] args) {  
        String s = "Java Programming";  
  
        System.out.println("Length = " + s.length());  
        System.out.println("Character at 2 = " + s.charAt(2));  
        System.out.println("Substring = " + s.substring(5));  
        System.out.println("Uppercase = " + s.toUpperCase());  
        System.out.println("Lowercase = " + s.toLowerCase());  
    }  
}
```

7. auto-boxing અને unboxing example સાથે સમજાવો.

Autoboxing શું છે?

Primitive data type ને automatically wrapper class object માં convert થવું autoboxing કહેવાય.

ઉદાહરણ:

`int → Integer`

Unboxing શું છે?

Wrapper class object ને automatically primitive type માં convert થવું unboxing કહેવાય.

ઉદાહરણ:

`Integer → int`

Autoboxing Example:

```
class Demo {  
    public static void main(String[] args) {  
        int a = 10;  
        Integer obj = a;    // autoboxing  
        System.out.println(obj);  
    }  
}
```

Output:

10

Unboxing Example:

```
class Demo {  
    public static void main(String[] args) {  
        Integer obj = 20;  
        int a = obj;    // unboxing  
        System.out.println(a);  
    }  
}
```

Output:

20

Combined Example:

```
class Demo {  
    public static void main(String[] args) {  
        int a = 50;  
        Integer obj = a;    // autoboxing  
  
        Integer x = 100;  
        int b = x;    // unboxing  
  
        System.out.println(obj);  
        System.out.println(b);  
    }  
}
```

ફાયદા:

- primitive અને object વચ્ચે સરળ conversion
- code concise બને છે

- collections સાથે કામ સરળ બને છે

8. Scanner class અને Command Line Arguments વડે user input કેવી રીતે લેવું તે સમજાવો.

A) Scanner class વડે input લેવું

Scanner class Java માં keyboard થી input લેવા માટે વપરાય છે.
આ માટે `java.util.Scanner` import કરવું પડે છે.

Example:

```
import java.util.Scanner;

class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

સમજાવટ:

- `nextLine()` → પૂરી line લે છે
- `nextInt()` → integer લે છે
- `nextFloat()` → float લે છે
- `next()` → single word લે છે

Scanner ની useful methods:

- `nextInt()`
- `nextFloat()`
- `nextDouble()`
- `nextLine()`
- `next()`
- `nextBoolean()`

B) Command Line Arguments વડે input લેવું

Command line arguments એ program run કરતી વખતે arguments pass કરવાના હોય છે.
આ arguments `String[] args` માં store થાય છે.

Example:

```
class CommandLineDemo {  
    public static void main(String[] args) {  
        System.out.println("First argument: " + args[0]);  
        System.out.println("Second argument: " + args[1]);  
    }  
}
```

જો run સમયે આવું આપીએ:

```
java CommandLineDemo Rahul 25
```

Output:

```
First argument: Rahul  
Second argument: 25
```

Command line arguments ની ખાસ વાત:

- બધા arguments string તરીકે જ મળે છે
- જરૂર પડે તો તેને `int` માં convert કરવું પડે

Example with conversion:

```
class CommandLineDemo {  
    public static void main(String[] args) {  
        int age = Integer.parseInt(args[0]);  
        System.out.println("Age = " + age);  
    }  
}
```
